

Nikhil Sharma

Pune, Maharashtra | +91 96610 52552 | nikhilsharmash20@gmail.com | [linkedin.com/in/nikhilsh2003](https://www.linkedin.com/in/nikhilsh2003) | github.com/nikhilsh2003

SKILLS

Languages: Java 17, Python, C++20, SQL

Backend Engineering: Spring Boot 3, REST APIs, Microservices, Hibernate, JPA, gRPC, API Design

Distributed Systems: Apache Kafka, Redis, Event-Driven Architecture, Distributed Transactions, Idempotency, Asynchronous Processing

AI & MCP Infrastructure: Model Context Protocol (MCP), RAG, LLM Tool-Calling, AI Governance, Prompt Filtering, AI Security Middleware

Cloud & Infrastructure: GCP, Docker, Kubernetes, Terraform, GitHub Actions, CI/CD

Data & Processing: MS SQL Server, BigQuery, Impala, MongoDB, TimescaleDB, Apache Spark, Apache Airflow, Cloud Composer, Dataproc

Security: OAuth2, JWT, RBAC, Microsoft Entra ID, SSO, SAML, OIDC, PII Anonymization, LLM Guardrails

Architecture & Patterns: System Design, HLD/LLD, JVM/G1GC, Connection Pooling, SQL Optimization, Strategy, Observer, Factory, Builder, Singleton, Decorator, SAGA, Circuit Breaker, Outbox

EDUCATION

NIT Warangal — B.Tech, Electronics & Communication Engineering CGPA 7.7/10 | May 2023

Codeforces Specialist (1418) | LeetCode 1904 | 800+ DSA problems solved | Global Rank 1, GFG Interview Series | Rank 1748, Google KickStart

EXPERIENCE

Software Engineer II — Backend Platform & AI Security — Deutsche Bank Nov 2025 – Present | Pune, India

- **AI Security Middleware:** Built AI security middleware for enterprise LLM workflows using PII anonymization, prompt filtering, and configurable guardrails, securing 5+ internal applications.
- **Latency Optimization:** Reduced AI inference latency from ~1s to under 200ms using Redis-based caching and query optimization, improving request throughput 5x.
- **MCP API Gateway:** Architected an MCP API Gateway using Microsoft Entra ID authentication and RBAC authorization, securing 100+ tool integrations against unauthorized invocation.
- **Database MCP & RAG:** Built an MCP server connecting internal metadata, transaction-status, Impala, and SQL Server data sources, enabling LLM-driven contextual query generation across financial data.
- **Code-Debugging MCP:** Built a code-debugging MCP that traces production errors to root cause and auto-generates pull requests with fix justification, adopted by the SL3 production-support team to reduce RCA and SLA resolution time.
- **AI Governance:** Partnered with 3+ platform and security teams to enforce AI guardrails across internal and external MCP integrations.
- **Mentorship:** Mentored 2 junior engineers on backend development and secure API design.

Software Engineer — Backend Platform & Data Engineering — Deutsche Bank Jul 2023 – Nov 2025 | Pune, India

- **Schema Migration:** Built a zero-downtime schema migration framework using dual-write/shadow-read orchestration and Kafka replay validation, cutting migration cycles from days to hours with zero data loss.
- **API Performance:** Re-architected high-traffic Spring Boot APIs by tuning HikariCP connection pooling, G1GC configuration, and SQL execution plans, achieving a 35% latency reduction and cutting SLO breaches from 90% to under 5%.
- **RIBB Reconciliation:** Automated reconciliation for the Risk in Banking Book pipeline covering ~\$100B in transaction value, cutting manual adjustment effort by 60%.
- **Cloud Migration:** Migrated on-premise batch pipelines to GCP using BigQuery, Dataproc, Cloud Composer, and Terraform, eliminating 80% of infrastructure overhead.
- **Pipeline Automation:** Automated retry handling using Airflow-managed workflows, reducing manual pipeline operations by 70%.
- **Secure Microservices:** Built RESTful Spring Boot microservices using OAuth2/JWT authentication and fine-grained authorization, supporting 100+ analysts in balance-sheet and liquidity workflows.

Software Development Engineer Intern — Data Quality Platform — Mastercard May 2022 – Jul 2022 | Pune, India

- **Data Quality Engine:** Built a configurable rule-based data quality validation engine in ASP.NET Core with a real-time monitoring dashboard, reducing downstream data incidents flagged by QA.

PROJECTS

QuantCore — Distributed High-Frequency Trading Engine Java 17, C++20, Spring Boot 3, Kafka, Redis [GitHub](#)

- **Matching Engine:** Engineered a deterministic C++20 order-matching engine using red-black trees and lock-free queues, achieving sub-500 microsecond latency and 10,000+ TPS under sustained synthetic load.
- **Distributed Backend:** Built a Spring Boot API gateway using OAuth2/JWT authentication and asynchronous Kafka order routing, deployed on Kubernetes via Terraform CI/CD.